

Lecture 5,6

1. Introduction to Containers

Containers are lightweight environments used to run applications.

They are **not virtual machines**—they are simply **processes** using the host's Linux kernel.

Key technologies that make containers possible

- **Namespaces:** isolate what a process can see (processes, network, filesystem, hostname).
 - **Cgroups:** control and limit what a process can *use* (CPU, RAM, network).
 - **Chroot:** isolates filesystem directories.
 - **OverlayFS:** layered filesystem where container images are read-only, and changes go to an upper writable layer.
-

2. Docker Basics

A Docker container consists of:

1. **Docker Engine** – runs containers.
2. **Docker Image** – blueprint for a container.
3. **Docker Hub** – registry storing images.

How images work

- Images are built from multiple **read-only layers**.
 - When a container writes, Docker uses **Copy-on-Write** (CoW).
-

3. Networking Fundamentals

Before understanding Docker networking, we need basic networking concepts:

IP Address

An IP is a numerical label (example: 192.168.1.10) used to identify devices.

CIDR Notation

Defines the network size, ex:

- 192.168.1.0/24 → 256 IPs
- 10.0.0.0/8 → 16 million IPs

Private IP Ranges

Used inside LAN networks:

- 10.0.0.0/8
 - 172.16.0.0/12
 - 192.168.0.0/16
-

4. Docker Networking

Docker supports many network drivers. Each provides different capabilities.

4.1 Bridge Network (default)

- Used when containers run on the **same host**.
- The default network is named **bridge**.
- Containers receive private IPs and can communicate within the same bridge.

User-defined Bridge

- Containers can communicate by **names**, not just IP.
- Provides better **isolation** between unrelated containers.
- Containers can be connected/disconnected **without restarting**.

Port Mapping

Example:

-p 8080:80 → Host port 8080 maps to container port 80.

4.2 Host Network

- Container uses the host's network stack directly.
 - No IP isolation (container shares host IP).
 - Useful for performance or applications needing many ports.
 - Works only on Linux.
-

4.3 Overlay Network

- Connects containers running on **different hosts**.
- Used in Docker Swarm or multi-host setups.

- Traffic may be encrypted using **IPSEC**.
-

4.4 Macvlan & IPvlan

Used when containers must appear as **real physical machines** on the network.

Use cases:

- Legacy applications
 - Network monitoring tools
 - Systems requiring direct layer-2 access
-

4.5 None Network

- Disables networking completely for the container.

4.6 Third-party Network Drivers

- External plugins providing specialized networking features.
-

5. Kubernetes Overview

Kubernetes is a **container orchestrator**.

It manages containerized applications across multiple servers.

5.1 Architecture

Kubernetes has two major parts:

Control Plane (Master)

- **API Server** – entry point of the cluster.
- **Scheduler** – decides which node runs a pod.
- **etcd** – distributed key-value store containing cluster state.

Worker Nodes

Each node runs:

- **Kubelet** – starts/monitors containers.
 - **Kube-proxy** – manages network rules.
 - **Container Runtime** (containerd, CRI-O, Docker).
-

6. Kubernetes Pods

- Smallest unit in Kubernetes.
- Pod = one or more containers deployed together.
- Containers inside a pod share:
 - Same IP
 - Same network namespace
 - Same storage

You said:

with a little translation to arabic

ChatGPT said:

Here is the **full, clean summary with a little Arabic translation** (simple and short Arabic lines to help understanding).

You can **copy–paste directly into Word**.



Full Summary: Docker, Kubernetes & Networking (With Arabic Notes)

1. Introduction to Containers

Containers are lightweight environments used to run applications.

They are **not virtual machines**—they are simply processes running on the host's Linux kernel.

VMs. الحاويات مجرد عمليات خفيفة تشترك في نواة لينكس، مش أجهزة كاملة زي الـ

Technologies that create containers

- **Namespaces:** isolate what a process can see (network, PIDs, filesystem).
Namespaces تعزل رؤية العملية.
 - **Cgroups:** limit and control resource usage (CPU, RAM, network).
Cgroups تحدد استهلاك الموارد.
 - **Chroot:** isolates the filesystem root directory.
Chroot يعزل ملفات النظام.
 - **OverlayFS:** layered filesystem; changes go to the top writable layer.
OverlayFS يعمل بنظام الطبقات.
-

2. Docker Basics

A Docker container consists of:

1. **Docker Engine** – runs containers.
2. **Docker Image** – blueprint containing files and instructions.
3. **Docker Hub** – registry to store images.
دوركر يشغل الحاويات باستخدام صور جاهزة

Image Layers

- Images are made of **read-only layers**.
- When a container writes, Docker uses **Copy-on-Write**.
التعديل لا يغير الصورة الأصلية، لكن يضيف طبقة فوقها

3. Networking Fundamentals

Before learning Docker networking, you must understand basic IP concepts.

IP Address

A numerical label such as 192.168.1.10.

يحدد كل جهاز في الشبكة IP عنوان

CIDR Notation

Defines network size:

- 192.168.1.0/24 → 256 IPs
- 10.0.0.0/8 → 16M IPs
bits. / يحدد حجم الشبكة بعدد الـ CIDR

Private IP Ranges

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16
دي الشبكات الداخلية فقط

4. Docker Networking

Docker supports several network drivers with different purposes.

4.1 Bridge Network (Default)

- Default network created when Docker starts.
- Containers receive private IP addresses.
- Containers can communicate on the same bridge network.

Bridge هو الوضع الافتراضي لربط الحاويات داخل نفس الجهاز.

User-Defined Bridge

- Enables **DNS by container name**.
 - Improves isolation between applications.
 - You can connect or disconnect containers at runtime.
- IP** الشبكة المخصصة تسمح بالاتصال بالاسم بدل الـ

Port Mapping

-p 8080:80 → host:container

ربط بورت من الجهاز إلى بورت داخل الحاوية

4.2 Host Network

- Container uses host network directly (same IP).
 - No network isolation.
 - High performance; useful for apps needing many ports.
 - Linux only.
- الحاوية تستخدم شبكة الجهاز مباشرة بدون عزل

4.3 Overlay Network

- Connects containers across **multiple hosts**.
 - Uses VXLAN for communication.
 - Supports encryption (IPSEC).
- شبكة توصل حاويات على أجهزة مختلفة

4.4 Macvlan & IPvlan

Used when a container must appear like a **physical machine** on the network.

Use cases:

- Legacy systems

- Network monitoring
 - Tools that need direct L2 access
حقيقي MAC تجعل الحاوية كأنها جهاز مستقل بعنوان
-

4.5 None Network

- Completely disables networking for the container.
بدون أي شبكة
-

4.6 Third-Party Drivers

Plugins providing special networking capabilities.
إضافات خارجية للشبكات

5. Kubernetes Overview

Kubernetes is a **container orchestrator** that manages containerized applications.
هو منسق للحاويات يعمل على إدارة التطبيقات Kubernetes

5.1 Kubernetes Architecture

Kubernetes contains two main parts:

Control Plane (Master)

Responsible for cluster decisions:

- **API Server** – front end of the cluster
- **Scheduler** – assigns Pods to nodes
- **etcd** – distributed database storing cluster state
الماستر مسؤول عن القرارات وحالة النظام

Worker Nodes

Run applications:

- **Kubelet** – runs and monitors Pods
 - **Kube-proxy** – manages network rules
 - **Container runtime** – runs containers
النودز تشغل التطبيقات فعلياً
-

6. Kubernetes Pods

- The smallest deployable unit in Kubernetes.
 - Pod = one or more containers.
 - Containers inside a Pod share:
 - the same IP
 - the same network namespace
 - the same storage (volumes)
- localhost. والـ IP البود مجموعة حاويات تشارك نفس الـ
-

7. Kubernetes Networking

- Every Pod gets its **own unique IP**.
 - Pods communicate **without NAT**, even across nodes.
 - Uses overlay networks (like VXLAN).
- NAT. البودات تتواصل مباشرة بدون

Kubernetes Services

Provide stable access to Pods.

Types:

- **ClusterIP** – internal only
 - **NodePort**
 - **LoadBalancer**
 - **Ingress** (Layer 7 routing)
- Service ثابت حتى لو تغيرت الحاويات
-

8. Load Balancing

Algorithms:

- **Round Robin** – rotate requests
 - **Least Connections** – send to least busy server
 - **IP Hash** – same client goes to same backend
- طرق مختلفة لتوزيع الحمل

Layers:

- **Layer 4** – TCP/UDP routing
 - **Layer 7** – HTTP/HTTPS smart routing
أذكى L7 أسرع، L4
-

9. Container Migration

Moving containers across nodes.

Types

- **Cold Migration:** stop → move → start
وقت توقف كبير.
- **Live Migration (CRIU):**
 - Freeze container
 - Copy memory + state
 - Resume on new machine
نقل الحاوية بدون توقف تقريباً.
- **Velero:** backup/restore for Kubernetes.
- **Databases:** use replication instead of migration.